Algorithms and Data Structures (ADS2)

Assessed Exercise 2

This exercise is for submission using Moodle and counts for 10% of the total assessment for the course.

The exercise is worth 20 points.

Deadline: Friday 20 March 2026 at 4:30pm.

Exercise overview

The exercise consists of two parts covering Dynamic Sets and Hashing. You must not rely on Java library classes in your implementation.

Submission

Submit a zip archive containing the Java sources of your implementations **and** a short (max 3 pages) report describing your approach for each part of the exercise.

Ensure the report is in PDF format and that none of your source files are password-protected.

Part 1: Dynamic Set ADT

A *Dynamic Set* is an abstract data type that stores distinct elements, without imposing an order. The ADT defines the following five main operations:

- ADD(S,x): add element x to S, if it is not already present
- REMOVE(S,x): remove element x from S, if it is present
- IS-ELEMENT(S,x): check whether element x is in set S
- SET-EMPTY(S): check whether set S has no elements
- SET-SIZE(S): return the number of elements of set S

The ADT also defines the following set-theoretical operations:

- UNION(S,T): return the union of sets S and T
- INTERSECTION (S,T): return the intersection of sets S and T
- DIFFERENCE(S,T): returns the difference of sets S and T
- SUBSET(S,T): check whether set S is a subset of set T

a) Implement in Java the main Dynamic Set operations above using a binary search tree. You may use a self-balancing tree but no extra marks will be awarded. [4]
b) Implement the set-theoretic operations. [4]
c) State and explain worst-case and average-case running times of all the operations implemented in a) and b). [2]

Part 2: Hashing

Consider the following hash function Java implementation:

```
public int hashCode(String s) {
   int hash = 0;
   int skip = Math.max(1, s.length() / 3);
```

```
    for (int i = 0; i < s.length(); i += skip)
        hash = (hash * 37) + s.charAt(i);
    return hash;
}
```

a) Explain the effect of the rule `skip = max(1, s.length()/3)` and the multiplicative factor `37`. [2]

b) Design and describe in you report an algorithm that ensures identical character samples at the skip positions, or controlled variation that yields identical polynomial hash contributions. Assume ASCII characters. [4]

c) Implement in Java a function that takes an integer $n \geq 2$ and generates n different strings that all hash to the same value when the hash function above is used. [4]